MIVA OPEN UNIVERSITY

DEPARTMENT OF CYBER SECURITY
COS 201: COMPUTER PROGRAMMING I

Student Course Management System

Mid-Semester Laboratory Assessment
Lab Report

STUDENT NAME	**ABAH ANTHONY**
MATRIC NUMBER	**30166082**
LEVEL	**200 LEVEL**
DEPARTMENT	**CYBER SECURITY**
COURSE	**COS 201 (Semester 1)**
SOURCE CODE	**https://github.com/UncleT-cyber/COS201-StudentCourseManagement**

October 2026

© Copyright 2025 MIVA Open University. All Rights Reserved

MIVA OPEN UNIVERSITY

DEPARTMENT OF CYBER SECURITY · COS 201 COMPUTER PROGRAMMING I · MID-SEMESTER LABORATORY ASSESSMENT

Student Course Management System Lab Report

NAME **ABAH ANTHONY** MATRIC NO **30166082** LEVEL **200 LEVEL** DATE **October 2026**

1. What the system does

The Student Course Management System is a desktop program that lets a student manage the courses carried in a semester. After signing in with a matric number, the student enrols in courses from the catalogue, sees all enrolled courses in one table, searches by course code, and watches the total credit units build up against the 24-unit limit. The course list is saved to a plain text file (`courses.txt`) and loaded back the next time the program starts, so nothing is lost between sessions. The program is written in Java using only the standard library (Swing), so it compiles with plain `javac` and needs no external libraries. The code is organised into four packages, so each part has one job: `model` holds the data, `core` holds the rules, recursion and file handling, `ui` is the interface, and `test` is the self-test.

2. How each major feature works

Feature	How it works
1. Add a course	The Enrolment form passes the typed code, title and units to `CourseManager.addCourse()`, which checks the code is a known course, the title is not empty, the units are a number, the 24-unit limit is not exceeded and prerequisites are met; only then is the `Course` object added to the list.
2. View all courses	The Home screen table is filled by looping over the enrolment list and printing each course's code, title, units and status. It refreshes automatically after every add or drop through the manager's change listener.
3. Search by code	The search box calls `searchCatalogue(code)`. The typed text is trimmed, upper-cased and stripped of spaces (`Course.normalizeCode()`), so `cos 201` and `COS 201` find the same course; the comparison itself is the recursive search in section 4.
4. Compute total units	`totalUnits()` runs the recursive total over the enrolment list and drives the credit progress bar (e.g. 13 of 24 units, 54%). It is recalculated after every enrolment and drop.
5. Save to file	`CourseFileIO.saveEnrollments()` writes one course per line to `courses.txt` with `|` between fields, using a `BufferedWriter` in try-with-resources so the file is always closed. Saving also happens automatically on exit.
6. Load from file	`loadEnrollments()` reads the file back at start-up and from the Load button. A line that cannot be parsed is counted and skipped instead of stopping the load; a missing file falls back to the seeded catalogue.
7. Exit	Logout / Exit saves the course list, stops the clock and autosave timers, then closes the window, so the file on disk is always up to date.

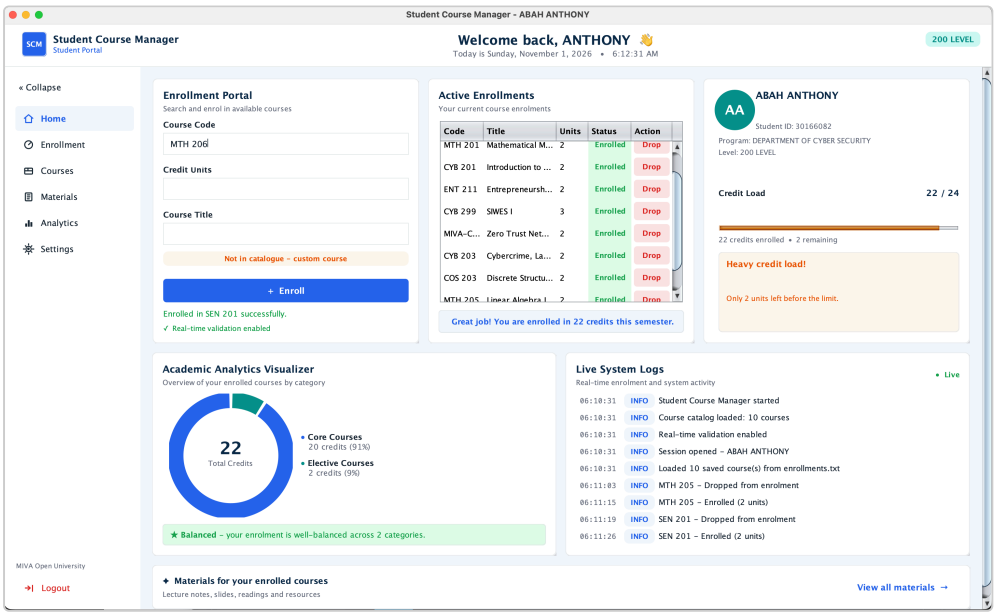


Figure 1: The program running, signed in to the Student Course Management System dashboard.

3. Challenges encountered and how errors were handled

- **Invalid student input.** Every field can be typed wrongly, so validation runs before anything touches the data. A non-numeric credit value raises `NumberFormatException`, caught in the form and shown as a message. Empty titles, unknown codes and breaking the 24-unit limit raise the custom *checked* exception `InvalidCourseException`, thrown by `addCourse()` *before* the list is changed, so a rejected enrolment never leaves a half-updated record. The interface only catches and displays, so the program never crashes on bad input. The login screen applies the same idea: regular expressions check the matric number and name before sign-in proceeds.
- **File errors.** Save and load use try-with-resources so handles are closed even if an operation fails. Malformed lines are skipped and counted rather than aborting the load, and a missing or unreadable file falls back to the seeded catalogue, so a damaged file cannot stop the program from starting.
- **Presenting the catalogue at any level.** The seed catalogue is written for 200 Level; supporting other levels without duplicating the data was solved with a regular-expression adaptor that shifts the level number in each code (SIWES courses are pinned and never shifted).

4. Where recursion was applied

Recursion is applied in two required places, both in `core/CourseManager.java`. **recursiveSearch** finds a course code: the *base case* is the index running past the end of the list (return `null`); the *recursive step* checks the current course and, if it is not a match, calls itself with `index + 1`. **recursiveTotalUnits** adds up the credit units: the base case returns `0` past the end, and the recursive step returns the current course's units plus the total of the rest. The public methods `searchCatalogue()` and `totalUnits()` start the recursion at index `0`.

> **Worked example, `recursiveTotalUnits` on [COS 201 (3 units), MTH 201 (2 units)]**
>
> ```
> total(0) = 3 + total(1)   course 0 is COS 201
> total(1) = 2 + total(2)   course 1 is MTH 201
> total(2) = 0              base case, the index is past the end
>
> total(1) = 2 + 0 = 2      the calls unwind
> total(0) = 3 + 2 = 5 units
> ```

All source code files are submitted through the repository: https://github.com/UncleT-cyber/COS201-StudentCourseManagement. The compiled program `SCM.jar` is attached to the repository's *Releases* page and runs on any computer with Java 8 or newer (`java -jar SCM.jar`). The project was built with plain `javac` and `java`, and its self-test passes **25 of 25 checks**.

ABAH ANTHONY, 30166082
Department of Cyber Security, MIVA Open University